



# Chương 6

## TỔNG HỢP CẤP CAO TRONG SoC FPGA

### System-on-Chip Design

# Mục lục

**5.1. GIỚI THIỆU**

**5.2. ĐÁNH GIÁ TÀI NGUYÊN SỬ DỤNG**

**5.3. ĐÁNH GIÁ HIỆU NĂNG CỦA HỆ  
THỐNG**

**5.4. ĐÁNH GIÁ HIỆU QUẢ CỦA HỆ THỐNG**

**5.5. ĐỘ TIN CẬY**

**5.6. SỰ ĐÁNH ĐỔI TRONG HỆ THỐNG SOC**

**5.7. TÍCH HỢP HỆ THỐNG**

# GIỚI THIỆU

- Tổng hợp cấp cao (HLS) mang lại nhiều lợi ích quan trọng trong thiết kế SoC trên FPGA, đặc biệt khi hệ thống ngày càng phức tạp và cần thời gian phát triển nhanh.
- HLS cho phép sử dụng ngôn ngữ cấp cao như Python, C/C++ thay vì Verilog hoặc VHDL, giảm đáng kể độ phức tạp phát triển. Điều này là cần thiết cho các ứng dụng cần tích hợp xử lý tín hiệu, quản lý bộ nhớ, giao tiếp ngoại vi, và thuật toán tính toán hiệu suất cao.
- HLS hiện nay tự động chuyển đổi mã cấp cao và hỗ trợ chuyển đổi các mô hình AI từ Python (như TensorFlow, PyTorch) sang RTL. Điều này giúp triển khai trực tiếp mô hình AI trên FPGA với hiệu suất cao và độ trễ thấp, tích hợp dễ dàng vào thiết kế SoC cho các ứng dụng như nhận diện hình ảnh và xử lý ngôn ngữ tự nhiên.
- Ngoài ra, HLS hỗ trợ tối ưu hóa như song song hóa và pipelining, tăng tốc độ thực thi và cải thiện hiệu suất hệ thống. Kỹ sư có thể tập trung vào giải quyết vấn đề thiết kế hệ thống, tăng năng suất và rút ngắn thời gian ra thị trường, đáp ứng nhu cầu cao về hệ thống AI và SoC trên FPGA.

# Giới thiệu Vivado HLS và Intel HLS

**Bảng 6.1: So sánh các đặc trưng chính giữa hai bộ công cụ Vivado HLS và Intel HLS**

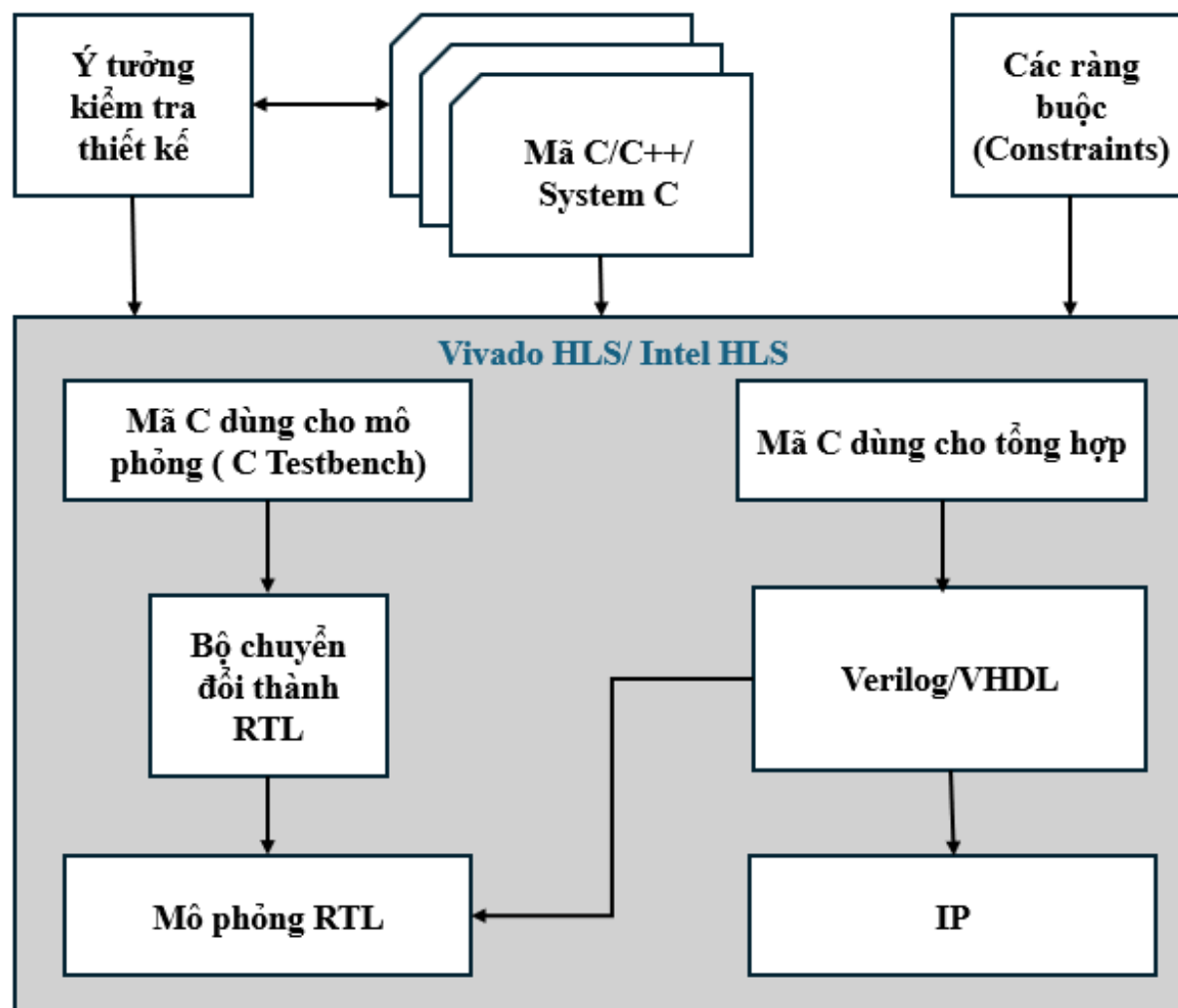
| Tiêu chí                         | Vivado HLS (Vitis HLS)                                                     | Intel HLS                                                               |
|----------------------------------|----------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Nhà phát triển                   | Xilinx (nay là một phần của AMD)                                           | Intel                                                                   |
| Tên hiện tại                     | Vitis HLS                                                                  | Intel® HLS Compiler                                                     |
| Mục tiêu thiết bị                | FPGA dòng Xilinx (Zynq, Virtex, Artix, Kintex)                             | FPGA dòng Intel (Stratix, Arria, Cyclone)                               |
| Ngôn ngữ đầu vào                 | C/C++, SystemC                                                             | C/C++, SystemC                                                          |
| Khả năng tích hợp với mô hình AI | Có thể tích hợp mô hình AI qua các công cụ như Vitis AI                    | Hỗ trợ chuyển đổi trực tiếp mô hình AI từ TensorFlow, PyTorch thành RTL |
| Môi trường tích hợp              | Tích hợp chặt chẽ với Vivado Design Suite và Vitis Unified Platform        | Tích hợp chặt chẽ với Intel® Quartus® Prime                             |
| Thư viện hỗ trợ                  | Thư viện đa dạng như DSP, xử lý video, fixed-point, floating-point         | Hỗ trợ các thư viện tính toán khoa học, fixed-point, floating-point     |
| Tối ưu hóa thiết kế              | Hỗ trợ phân giải vòng lặp (loop unrolling), pipeline, và tối ưu hóa bộ nhớ | Tương tự, với các khả năng tối ưu hóa như pipeline, loop unrolling      |

# Giới thiệu Vivado HLS và Intel HLS

**Bảng 6.1: So sánh các đặc trưng chính giữa hai bộ công cụ Vivado HLS và Intel HLS**

| Tiêu chí                         | Vivado HLS (Vitis HLS)                                                                   | Intel HLS                                                                           |
|----------------------------------|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Công cụ mô phỏng và debug        | Hỗ trợ mô phỏng dạng sóng và kiểm lỗi với Vivado Simulator                               | Hỗ trợ mô phỏng dạng sóng và kiểm lỗi tích hợp                                      |
| Khả năng đa nền tảng             | Hỗ trợ trên Windows và Linux                                                             | Hỗ trợ trên Windows và Linux                                                        |
| Ứng dụng chính                   | Tăng tốc các ứng dụng nhúng như DSP, xử lý hình ảnh, video, AI                           | Tăng tốc các ứng dụng DSP, HPC, AI, và nhúng                                        |
| Mức độ sử dụng trong công nghiệp | Phổ biến trong các hệ thống nhúng, đặc biệt với SoC dòng Zynq                            | Phổ biến trong các ứng dụng HPC và AI yêu cầu hiệu năng cao                         |
| Hỗ trợ hệ sinh thái              | Là một phần của hệ sinh thái Xilinx, kết hợp chặt chẽ với các công cụ Vitis AI và Vivado | Là một phần của hệ sinh thái Intel, tích hợp tốt với các công cụ OneAPI và OpenVINO |

# Quy trình tạo IP từ công cụ HLS



Hình 6.9: Quy trình tạo IP tự thiết kế với công cụ Vivado HLS/Intel HLS

# Quy trình tạo IP từ công cụ HLS

- **Đầu vào của một project trong quy trình tổng hợp cấp cao gồm:**
  - Hàm chính (top function) được viết bằng ngôn ngữ C, C++, hoặc SystemC: đây là đầu vào chính cho Vivado HLS hoặc Intel HLS. Hàm này có thể chứa một hệ thống phân cấp của các hàm con.
  - Các ràng buộc (constraints): các ràng buộc là bắt buộc, bao gồm xung đồng hồ (clock), độ không đảm bảo của xung đồng hồ (uncertainly clock) và loại FPGA được sử dụng. Độ không đảm bảo của xung đồng hồ mặc định là 12,5% nếu không được chỉ định.
  - Chỉ thị (directives): chỉ thị là tùy chọn và chỉ đạo quá trình tổng hợp để thực hiện một hành vi cụ thể hoặc một kỹ thuật tối ưu hóa cụ thể.
  - Mã C dùng cho mô phỏng (C testbench) và những tệp liên quan: quy trình tổng hợp cấp cao sử dụng C testbench để mô phỏng hàm trước khi tổng hợp và kiểm thử đầu ra RTL nhờ vào quá trình mô phỏng C/RTL (C/RTL simulation).
  - Đầu ra của một project trong HLS gồm:
    - Tệp thiết kế RTL dưới dạng ngôn ngữ mô tả phần cứng: đây là đầu ra chủ yếu của một project dưới dạng các ngôn ngữ VHDL (IEEE 1076-2000) hoặc Verilog (IEEE 1364-2001). Vivado HLS đóng gói các tệp triển khai dưới dạng khối IP để sử dụng với các IP khác trong quy trình thiết kế của Xilinx.
    - Tệp báo cáo: báo cáo kết quả của quá trình tổng hợp, giả lập, đóng gói thành IP.

# Các lợi ích của phương pháp tổng hợp cấp cao

- Tổng hợp cấp cao là cầu nối giữa phần cứng và phần mềm, mang lại các lợi ích chính sau:
  - Cải thiện năng suất của các kỹ sư thiết kế phần cứng: Có thể làm việc ở mức trừu tượng cao hơn trong khi vẫn tạo ra được phần cứng có hiệu năng cao.
  - Cải thiện năng suất hệ thống cho các kỹ sư thiết kế phần mềm: Các kỹ sư thiết kế phần mềm có thể tăng tốc độ tính toán chuyên sâu của thuật toán trên mục tiêu biên dịch mới là FPGA.
  - Phát triển các thuật toán ở mức ngôn ngữ cấp trung như C: Làm việc ở mức độ trừu tượng thay vì quá trình thiết kế chi tiết tiêu tốn nhiều thời gian.
  - Phát triển các thuật toán ở mức ngôn ngữ cấp trung như C: Làm việc ở mức độ trừu tượng thay vì quá trình thiết kế chi tiết tiêu tốn nhiều thời gian.
  - Kiểm soát quá trình tổng hợp ngôn ngữ C thông qua các chỉ thị tối ưu (optimization directives): tạo ra các bản thiết kế phần cứng cụ thể có hiệu suất cao.
  - Tạo ra nhiều thiết kế từ ngôn ngữ C nhờ vào các chỉ thị tối ưu: mở rộng không gian thiết kế, làm tăng khả năng tìm được các thiết kế tối ưu nhất.
  - Tạo ra mã nguồn dễ đọc và tiện lợi: nhằm mục tiêu tái sử dụng vào các thiết bị hoặc dự án khác.



# Các kỹ thuật tối ưu hóa thiết kế với HLS

## ***Phân giải vòng lặp***

- *Khi không sử dụng phân giải vòng lặp ta có đoạn code sau:*

```
int sum = 0;
int array[8] = {1, 2, 3, 4, 5, 6, 7, 8};
for (int i = 0; i < 8; i++) {
    sum += array[i];
}
```

*Hình 6.10: Minh họa đoạn mã xử lý không sử dụng kỹ thuật phân giải vòng lặp*

- Đoạn mã C trong hình 6.10 sẽ thực hiện 8 lần lặp, mỗi lần cộng một phần tử vào biến sum.

# Các kỹ thuật tối ưu hóa thiết kế với HLS

## ***Phân giải vòng lặp***

- Áp dụng kỹ thuật phân giải vòng lặp, ta có thể giảm số lần lặp của vòng lặp bằng cách thực hiện nhiều phép cộng trong mỗi lần lặp:

```
int sum = 0;
int array[8] = {1, 2, 3, 4, 5, 6, 7, 8};
// Mỗi lần lặp cộng 2 phần tử, giảm số lần lặp xuống còn một nửa
for (int i = 0; i < 8; i += 2) {
    sum += array[i] + array[i+1];
}
```

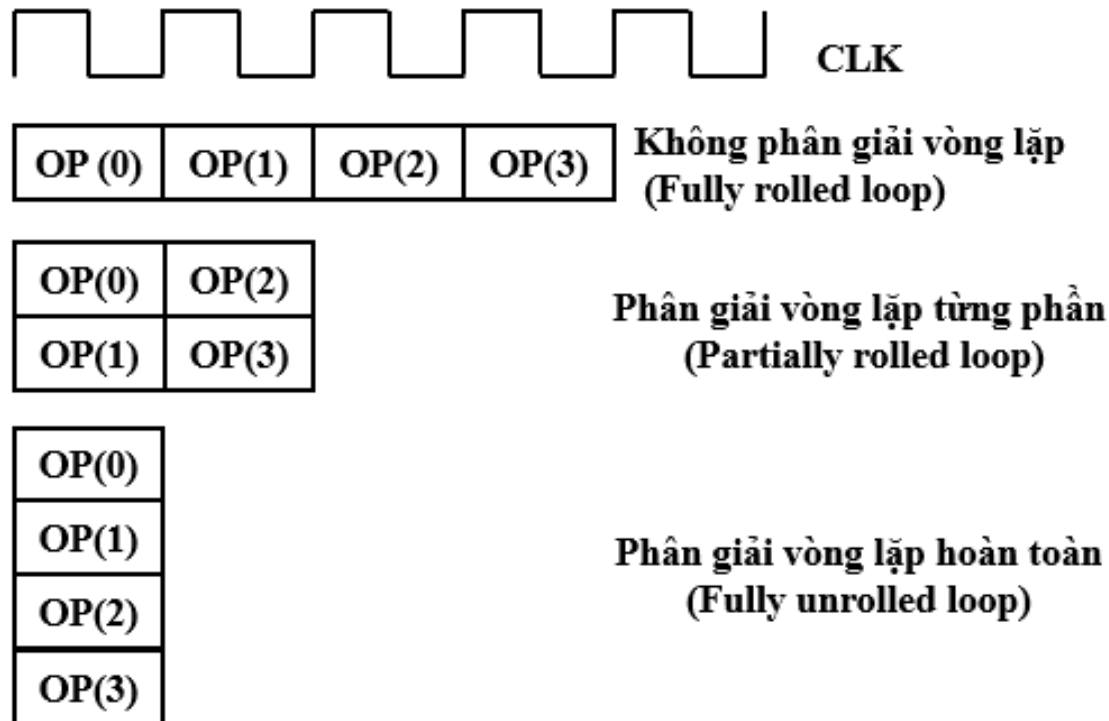
*Hình 6.11: Minh họa đoạn mã xử lý với kỹ thuật phân giải vòng lặp từng phần*

- Đoạn mã C trong hình 6.11 chỉ cần chạy 4 lần thay vì 8 lần, mỗi lần cộng 2 phần tử vào tổng sum điều này giảm độ phức tạp của vòng lặp và có thể cải thiện hiệu suất, đặc biệt là trong các ứng dụng xử lý dữ liệu lớn hoặc yêu cầu thời gian thực.

# Các kỹ thuật tối ưu hóa thiết kế với HLS

## Phân giải vòng lặp

- Kỹ thuật không phân giải vòng lặp (fully rolled loop), phân giải vòng lặp từng phần (partially rolled loop), và phân giải vòng lặp hoàn toàn (fully unrolled loop) qua thao tác thực hiện một nhiệm vụ lặp lại 4 lần.



Hình 6.12: Minh họa một số kỹ thuật phân giải vòng lặp.

# Các kỹ thuật tối ưu hóa thiết kế với HLS

## Lợi ích

- Tăng thông lượng: Cải thiện hiệu suất bằng cách thực hiện nhiều hơn một lần lặp trong mỗi chu kỳ.
- Giảm độ trễ: Giảm số lần kiểm tra điều kiện vòng lặp và số lần nhảy, giúp giảm độ trễ tổng thể.
- Hiệu suất tối ưu: Trong một số trường hợp, phân giải vòng lặp giúp tận dụng tốt hơn các nguồn lực phần cứng, như bộ nhớ cache hoặc đơn vị xử lý song song.

## Hạn Chế

- Tăng diện tích sử dụng: Trong thiết kế mạch, việc sử dụng nhiều sao chép của phần thân vòng lặp có thể dẫn đến tăng diện tích sử dụng.
- Quản lý phức tạp: Cần phải cân nhắc kỹ lưỡng mức độ unrolling phù hợp để tránh làm giảm hiệu quả do quá trình quản lý phức tạp hóa.
- Phụ thuộc vào kích thước vòng lặp: Hiệu quả của kỹ thuật này có thể phụ thuộc vào kích thước của vòng lặp và số lần lặp cố định.

# Các kỹ thuật tối ưu hóa thiết kế với HLS

## Phân chia mảng (Array Partitioning)

Ma trận dữ liệu

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  | 6  |
| 7  | 8  | 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 | 17 | 18 |
| 19 | 20 | 21 | 22 | 23 | 24 |
| 25 | 26 | 27 | 28 | 29 | 30 |
| 31 | 32 | 33 | 34 | 35 | 36 |

(a)

|    |    |    |
|----|----|----|
| 1  | 7  | 13 |
| 2  | 8  | 14 |
| 3  | 9  | 15 |
| 4  | 10 | 16 |
| 5  | 11 | 17 |
| 6  | 12 | 18 |
| 19 | 25 | 31 |
| 20 | 26 | 32 |
| 21 | 27 | 33 |
| 22 | 28 | 34 |
| 23 | 29 | 35 |
| 24 | 30 | 36 |

Phân chia mảng  
theo dạng khối

(b)

|    |    |    |
|----|----|----|
| 1  | 7  | 13 |
| 19 | 25 | 31 |
| 2  | 8  | 14 |
| 20 | 26 | 32 |
| 3  | 9  | 15 |
| 21 | 27 | 33 |
| 4  | 10 | 16 |
| 22 | 28 | 34 |
| 5  | 11 | 17 |
| 23 | 29 | 35 |
| 6  | 12 | 18 |
| 24 | 30 | 36 |

Phân chia mảng  
theo dạng đa chiều

(c)

|    |    |    |
|----|----|----|
| 1  | 2  | 3  |
| 4  | 5  | 6  |
| 7  | 8  | 9  |
| 10 | 11 | 12 |
| 13 | 14 | 15 |
| 16 | 17 | 18 |
| 19 | 20 | 21 |
| 22 | 23 | 24 |
| 25 | 26 | 27 |
| 28 | 29 | 30 |
| 31 | 32 | 33 |
| 34 | 35 | 36 |

Phân chia mảng  
theo chu kỳ

Hình 6.13: Minh họa các kỹ thuật phân chia mảng trong tổng hợp cấp cao

# Các kỹ thuật tối ưu hóa thiết kế với HLS

Trong tổng hợp cấp cao, *pragma* là một công cụ mạnh mẽ giúp các nhà thiết kế giao tiếp trực tiếp với bộ biên dịch tổng hợp cấp cao để hướng dẫn cách thức tổng hợp một đoạn mã cụ thể. *Pragma* không thay đổi logic của chương trình nhưng có thể ảnh hưởng đáng kể đến hiệu suất, diện tích, và công suất của thiết kế phần cứng kết quả. Chúng cho phép nhà thiết kế kiểm soát tối ưu hóa mà không cần thay đổi mã nguồn của thuật toán.

Các *pragma* thường được sử dụng trong tổng hợp cấp cao bao gồm:

- **#pragma HLS pipeline:** Tối ưu hóa vòng lặp bằng cách pipelining, giúp giảm độ trễ và tăng thông lượng bằng cách cho phép nhiều lần lặp của vòng lặp thực hiện đồng thời ở các giai đoạn khác nhau của việc xử lý.
- **#pragma HLS unroll:** Giải nén vòng lặp để thực hiện song song, cải thiện tốc độ thực thi nhưng chi phí sử dụng tài nguyên phần cứng nhiều hơn.
- **#pragma HLS array\_partition:** Chỉ định cách một mảng được phân chia thành các phần nhỏ hơn, cho phép truy cập đồng thời nhiều phần tử, điều này rất hữu ích trong các thiết kế yêu cầu xử lý song song cao.
- **#pragma HLS allocate:** Kiểm soát việc phân bổ tài nguyên phần cứng cho các biến hoặc mảng, giúp tối ưu hóa việc sử dụng tài nguyên.
- **#pragma HLS inline:** Chỉ định rằng một hàm nên được inline, tức là mã của hàm được chèn trực tiếp vào nơi gọi hàm, có thể giúp giảm độ trễ và tối ưu hóa hiệu suất.

# Các kỹ thuật tối ưu hóa thiết kế với HLS

## *Pragma trong tổng hợp cấp cao*

```
#include <ap_int.h>
#define DATA_SIZE 256
#define KERNEL_SIZE 3
// Giả sử sử dụng fixed-point integer 8 bit cho dữ liệu và kernel
typedef ap_int<8> data_t;
typedef ap_int<8> kernel_t;
// Hàm tính convolution
void convolution(data_t input[DATA_SIZE], kernel_t kernel[KERNEL_SIZE],
               data_t output[DATA_SIZE]) {
    int i, j;
    // Áp dụng pragma HLS pipeline để tối ưu hóa việc thực hiện vòng lặp chính
    #pragma HLS PIPELINE
    for(i = 0; i < DATA_SIZE; i++) {
        data_t sum = 0;
        // Áp dụng pragma HLS unroll để tối ưu hóa việc tính toán convolution
        #pragma HLS UNROLL factor=2
        For (j = 0; j < KERNEL_SIZE; j++) {
            // Xử lý biên của mảng
            int idx = i + j - KERNEL_SIZE / 2;
            if (idx >= 0 && idx < DATA_SIZE) {
                sum += input[idx] * kernel[j];
            }
        }
        output[i] = sum;
    }
}
```

Hình 6.14: Minh họa đoạn mã xử lý có tích hợp các chỉ định pragma

# Các kỹ thuật tối ưu hóa thiết kế với HLS

- **Kỹ thuật lượng tử hóa (quantization)** là phương pháp tối ưu hóa quan trọng giúp giảm độ phức tạp tính toán và tài nguyên phần cứng. Giúp chuyển đổi giá trị dấu phẩy động sang số nguyên hoặc dấu chấm cố định, giảm kích thước bộ nhớ, tiêu thụ năng lượng và tăng tốc độ xử lý.
- Được sử dụng rộng rãi trong AI, xử lý tín hiệu số và hệ thống nhúng trên FPGA, đảm bảo hiệu suất và tính chính xác. Hỗ trợ kỹ thuật lượng tử hóa, cho phép tùy chỉnh độ chính xác, độ dài bit và định dạng dữ liệu.

Ví dụ minh họa cụ thể về kỹ thuật lượng tử hóa trong tổng hợp cấp cao thông qua bài toán nhân hai ma trận. Giả sử chúng ta cần thực hiện phép nhân hai ma trận  $4 \times 4$  trong phần cứng FPGA theo hướng tổng hợp cấp cao. Đầu tiên, thuật toán được viết bằng số dấu phẩy động như mô tả trong hình 6.15 cho các biến đầu vào và đầu ra khai báo với kiểu dữ liệu 'float', sau đó chúng ta áp dụng kỹ thuật lượng tử hóa để chuyển đổi sang số dấu chấm cố định với độ chính xác  $\langle 16,6 \rangle$  như trong hình 6.16. Ở đây các biến đầu vào đầu ra sẽ được khai báo với kiểu dữ liệu 'ap\_fixed  $\langle 16,6 \rangle$ ' với 6 bit dành cho phần thập phân và 10 bit dành cho phần nguyên nhằm tối ưu tài nguyên phần cứng.



# Các kỹ thuật tối ưu hóa thiết kế với HLS

## Kỹ thuật lượng tử hóa

```
#include <hls_stream.h>

void matrix_multiply(float A[4][4], float B[4][4], float C[4][4]) {
#pragma HLS PIPELINE
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            float sum = 0.0;
            for (int k = 0; k < 4; k++) {
                sum += A[i][k] * B[k][j];
            }
            C[i][j] = sum;
        }
    }
}
```

Hình 6.15: Mã C sử dụng dấu phẩy động để tính toán

# Các kỹ thuật tối ưu hóa thiết kế với HLS

## Kỹ thuật lượng tử hóa

```
#include <hls_stream.h>
#include <ap_fixed.h>

void matrix_multiply(ap_fixed<16,6> A[4][4], ap_fixed<16,6> B[4][4], ap_fixed<16,6> C[4][4])
#pragma HLS PIPELINE
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            ap_fixed<16,6> sum = 0;
            for (int k = 0; k < 4; k++) {
                sum += A[i][k] * B[k][j];
            }
            C[i][j] = sum;
        }
    }
}
```

Hình 6.16: Mã C sử dụng dấu chấm cố định để tính toán

# Các kỹ thuật tối ưu hóa thiết kế với HLS

## Kỹ thuật lượng tử hóa

**Bảng 6.2: Báo cáo tài nguyên phần cứng với dữ liệu đầu vào ở hai kiểu dữ liệu: dấu chấm cố định và dấu phẩy động.**

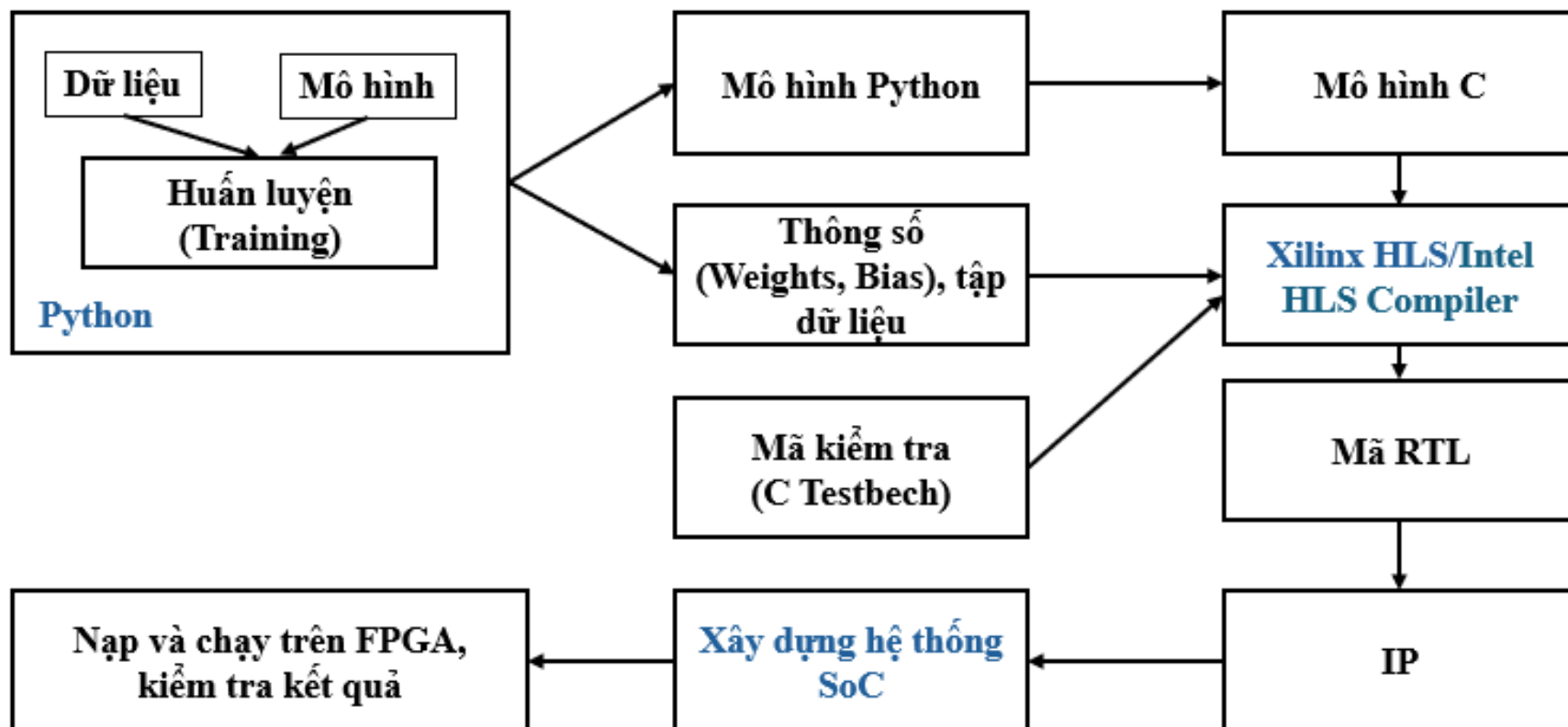
| Tài nguyên | Thiết kế 1(dấu chấp cố định) | Thiết kế 2 (dấu phẩy động) |
|------------|------------------------------|----------------------------|
| BRAM       | 0                            | 0                          |
| DSP        | 64                           | 28                         |
| FF         | 1829                         | 5326                       |
| LUT        | 425                          | 3281                       |
| URAM       | 0                            | 0                          |
| Độ trễ (s) | 32                           | 40                         |

# Quy trình thiết kế SoC theo phương pháp tổng hợp cấp cao

**Quy trình thiết kế thiết kế SoC theo phương pháp tổng hợp cấp cao được thực hiện thông qua các bước sau:**

- Bước 1: chuẩn bị dữ liệu gồm thu thập dữ liệu, xử lý dữ liệu ví dụ khử nhiễu, chuẩn hóa, loại bỏ các đột biến..., và xây dựng mô hình mạng tương ứng từ các nền tảng như Keras hoặc Pytorch sử dụng ngôn ngữ lập trình Python.
- Bước 2: huấn luyện mô hình đến khi mô hình hội tụ được các chuẩn đánh giá như mong muốn như độ chính xác (accuracy), chỉ số F1..., thì ta có được giá trị của các thông số (parameters) gồm weights, bias. Đây là những trọng số quan trọng trong một mô hình trí tuệ nhân tạo. Các trọng số này cũng sẽ lưu ở dạng file .dat hoặc file .txt.
- Bước 3: chuyển mô hình được viết bằng mã python sang mã C ở bước mạng lan truyền tới (forwarding). Đưa tất cả các tệp mã C và tệp trọng số vào công cụ tổng hợp cấp cao để chuyển qua mã RTL. Bước chuyển từ mã C sang mã RTL mang đậm tính chất phần cứng nên cần chú trọng các yêu cầu về cấu trúc phần cứng IP như bộ nhớ, số bit của dữ liệu, ... Do đó, ở giai đoạn này ta có thể tối ưu mã C sử dụng các phương pháp tối ưu như lượng tử hóa, phân giải vòng lặp, kỹ thuật pipeline, phân chia mảng....
- Bước 4: Mã RTL được tạo ra ban đầu chỉ có thể được sử dụng để kiểm tra chức năng của thiết kế trong phạm vi của một công cụ tổng hợp cấp cao cụ thể. Để mở rộng khả năng tương tác với các công cụ khác và tiến tới thiết kế chip mà không bị ràng buộc bởi bất kỳ công cụ nào, ta cần thiết lập một lớp giao tiếp tuân theo chuẩn bus phổ biến, chẳng hạn như AXI của Xilinx hoặc Avalon của Altera. Điều này cho phép dễ dàng tích hợp IP vừa thiết kế vào hệ thống SoC tương ứng.
- Bước 5: Sau khi hoàn tất việc kiểm tra chức năng bằng mã kiểm tra (testbench) và đóng gói IP với giao diện tuân theo chuẩn bus mong muốn, bước tiếp theo là tiến hành thiết kế hệ thống SoC và tích hợp IP vừa thiết kế vào hệ thống.
- Bước 6: kiểm tra kết quả sau khi thực hiện hệ thống trên FPGA. Ta có thể lấy kết quả được lưu ở dạng tệp .dat hoặc tệp .txt. sau đó so sánh trực tiếp với kết quả từ Python hoặc C, từ đó ta có thể đánh giá được mô hình đã hoàn thành yêu cầu đặt ra hay chưa.

# Quy trình thiết kế SoC theo phương pháp tổng hợp cấp cao



Hình 6.17: Quy trình thiết kế hệ thống từ HLS xuống SoC

# Ví dụ thiết kế SoC bằng phương pháp tổng hợp cấp cao

**Ví dụ tạo IP từ phương pháp tổng hợp cấp cao**

```

1  #include <hls_stream.h>
2  #include <ap_fixed.h>
3
4  void matrix_multiply(ap_fixed<16,6> A[4][4], ap_fixed<16,6> B[4][4], ap_fixed<16,6> C[4][4]) {
5      #pragma HLS PIPELINE
6      #pragma HLS INTERFACE mode=ap_memory port=A storage_type=ram_1p
7      #pragma HLS INTERFACE mode=ap_memory port=B storage_type=ram_1p
8      #pragma HLS INTERFACE mode=ap_memory port=C storage_type=ram_1p
9      for (int i = 0; i < 4; i++) {
10         for (int j = 0; j < 4; j++) {
11             ap_fixed<16,6> sum = 0;
12             for (int k = 0; k < 4; k++) {
13                 sum += A[i][k] * B[k][j];
14             }
15             C[i][j] = sum;
16         }
17     }
18 }

```

Hình 6.18: Minh họa đoạn mã nguồn để tạo IP nhân hai ma trận từ mã C++.

```

1  #include <iostream>
2  #include <hls_stream.h>
3  #include <ap_fixed.h>
4
5  void matrix_multiply(ap_fixed<16,6> A[4][4], ap_fixed<16,6> B[4][4], ap_fixed<16,6> C[4][4]);
6
7  int main() {
8      const int SIZE = 4;
9      ap_fixed<16,6> A[SIZE][SIZE] = {
10         {1.0, 2.0, 3.0, 4.0},
11         {5.0, 6.0, 7.0, 8.0},
12         {9.0, 10.0, 11.0, 12.0},
13         {13.0, 14.0, 15.0, 16.0}
14     };
15
16     ap_fixed<16,6> B[SIZE][SIZE] = {
17         {1.0, 0.0, 0.0, 0.0},
18         {0.0, 1.0, 0.0, 0.0},
19         {0.0, 0.0, 1.0, 0.0},
20         {0.0, 0.0, 0.0, 1.0}
21     };
22
23     ap_fixed<16,6> C[SIZE][SIZE] = {0};
24
25     matrix_multiply(A, B, C);
26
27     std::cout << "Result of matrix multiplication:" << std::endl;
28     for (int i = 0; i < SIZE; i++) {
29         for (int j = 0; j < SIZE; j++) {
30             std::cout << C[i][j].to_float() << " ";
31         }
32         std::cout << std::endl;
33     }
34
35     return 0;
36 }

```

Hình 6.19: Minh họa đoạn mã nguồn kiểm tra C++ dùng để kiểm tra chức năng bài toán nhân hai trận.

# Ví dụ thiết kế SoC bằng phương pháp tổng hợp cấp cao

Để kiểm tra chức năng của bài toán nhân hai ma trận:

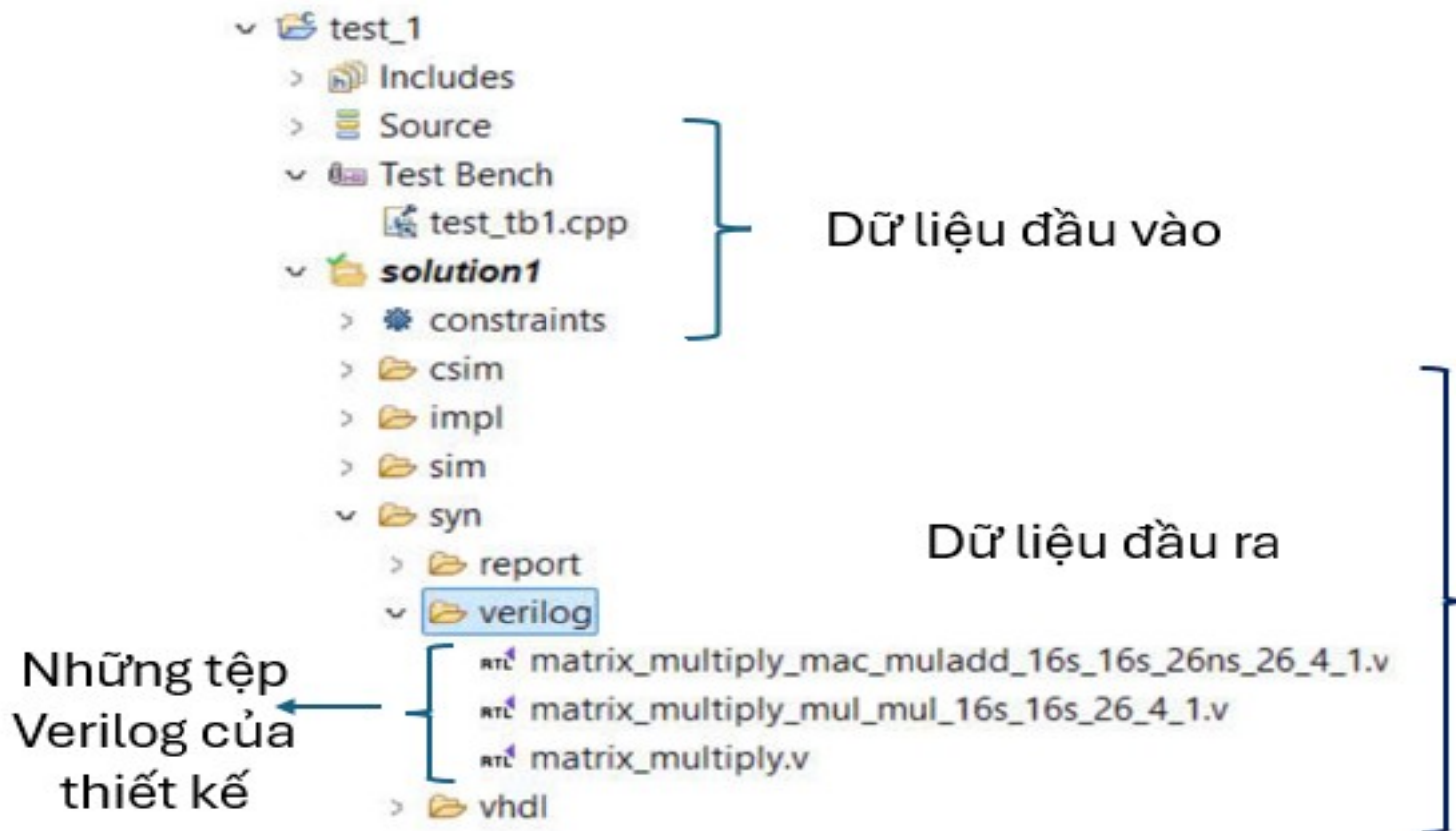
- Đoạn mã trong hình 6.19 thực hiện việc khai báo dữ liệu ban đầu cho hai ma trận A và B.
- Sau đó, hàm *matrix\_multiply* (đã được định nghĩa trước đó trong phần mã nguồn) được gọi vào, tương tự như cách gọi một chương trình con trong ngôn ngữ lập trình C thông thường.
- Cuối cùng, kết quả được kiểm tra bằng cách in các phần tử của ma trận C sau khi tính toán để xác minh tính đúng đắn của phép nhân.
- Tiếp theo, tần số hoạt động ban đầu của mạch được thiết lập thông qua việc định nghĩa các ràng buộc, như được mô tả trong hình 6.20.

```
# This constraints file contains default clock frequencies to be used during out-of-context flows such as
# OOC Synthesis and Hierarchical Designs. For best results the frequencies should be modified
# to match the target frequencies.
# This constraints file is not used in normal top-down synthesis (the default flow of Vivado)
create_clock -name ap_clk -period 10.000 [get_ports ap_clk]
```

Hình 6.20: Tạo tần số hoạt động ban đầu cho thiết kế.



# Ví dụ thiết kế SoC bằng phương pháp tổng hợp cấp cao

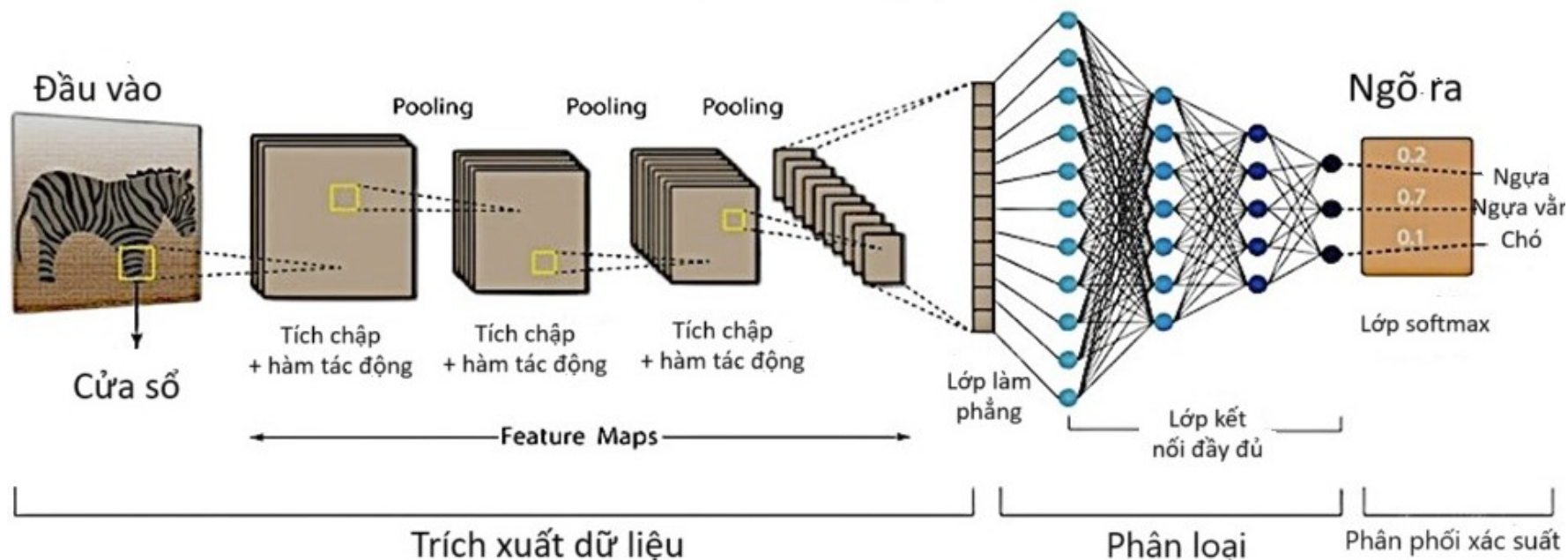


Hình 6.21: Tổ chức lưu trữ trong bộ cộng cụ Vitis HLS cho một dự án.

# Ví dụ thiết kế SoC bằng phương pháp tổng hợp cấp cao

## Mạng tích chập

Cấu trúc mạng tích chập (CNN)



Hình 6.22: Cấu trúc một mô hình mạng tích chập cơ bản

# Ví dụ thiết kế SoC bằng phương pháp tổng hợp cấp cao

Kích thước của ma trận đầu ra của lớp tích chập được xác định theo công thức:

$$w_0 * h_0 = \left( \frac{w_i + 2p - k}{s} + 1 \right) * \left( \frac{h_i + 2p - k}{s} + 1 \right) \quad (6.1)$$

Trong đó

$w_0$  và  $h_0$  là chiều rộng và chiều cao của ma trận đầu ra.

$w_i$  và  $h_i$  là chiều rộng và chiều cao của ma trận đầu vào.

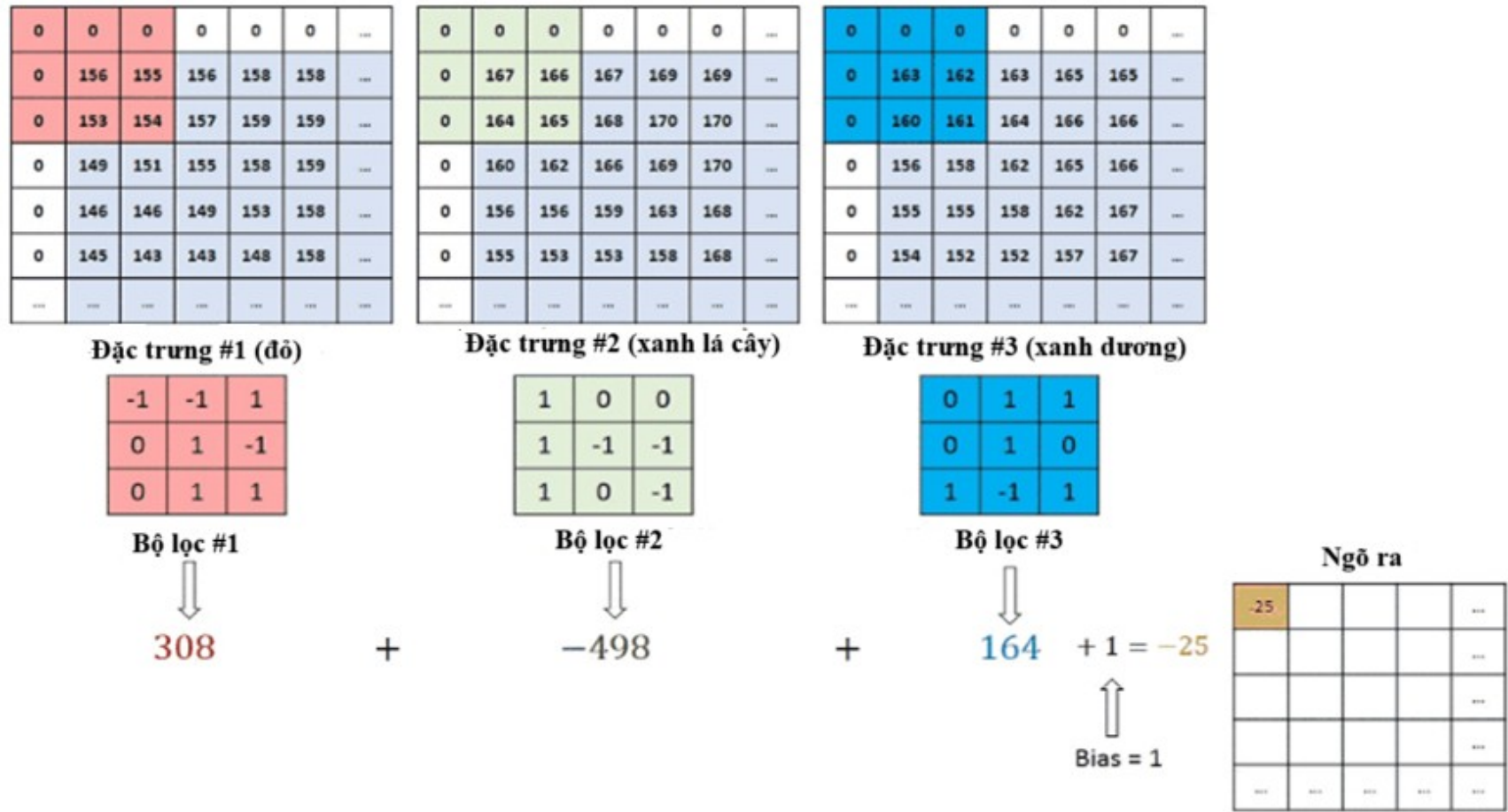
$p$  là kích thước đệm.

$s$  là khoảng cách giữa mỗi lần trượt của bộ lọc.

$k$  là kích thước của bộ lọc.

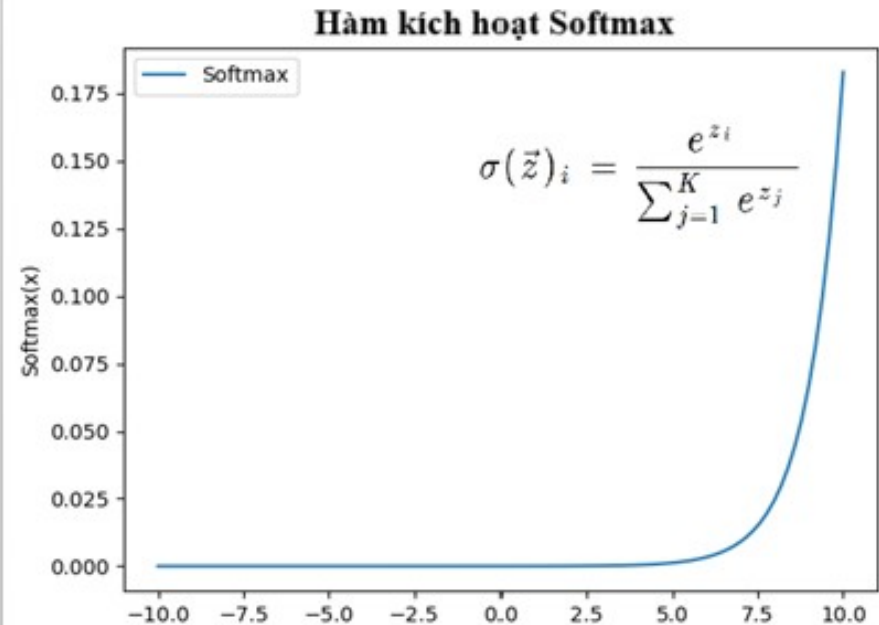
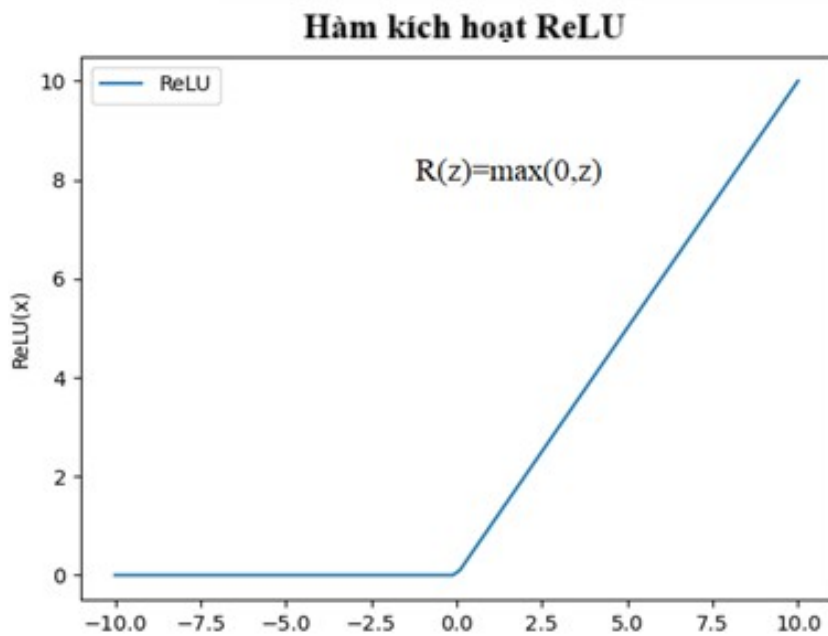
*Công thức tính kích thước của ma trận đầu ra*

# Ví dụ thiết kế SoC bằng phương pháp tổng hợp cấp cao



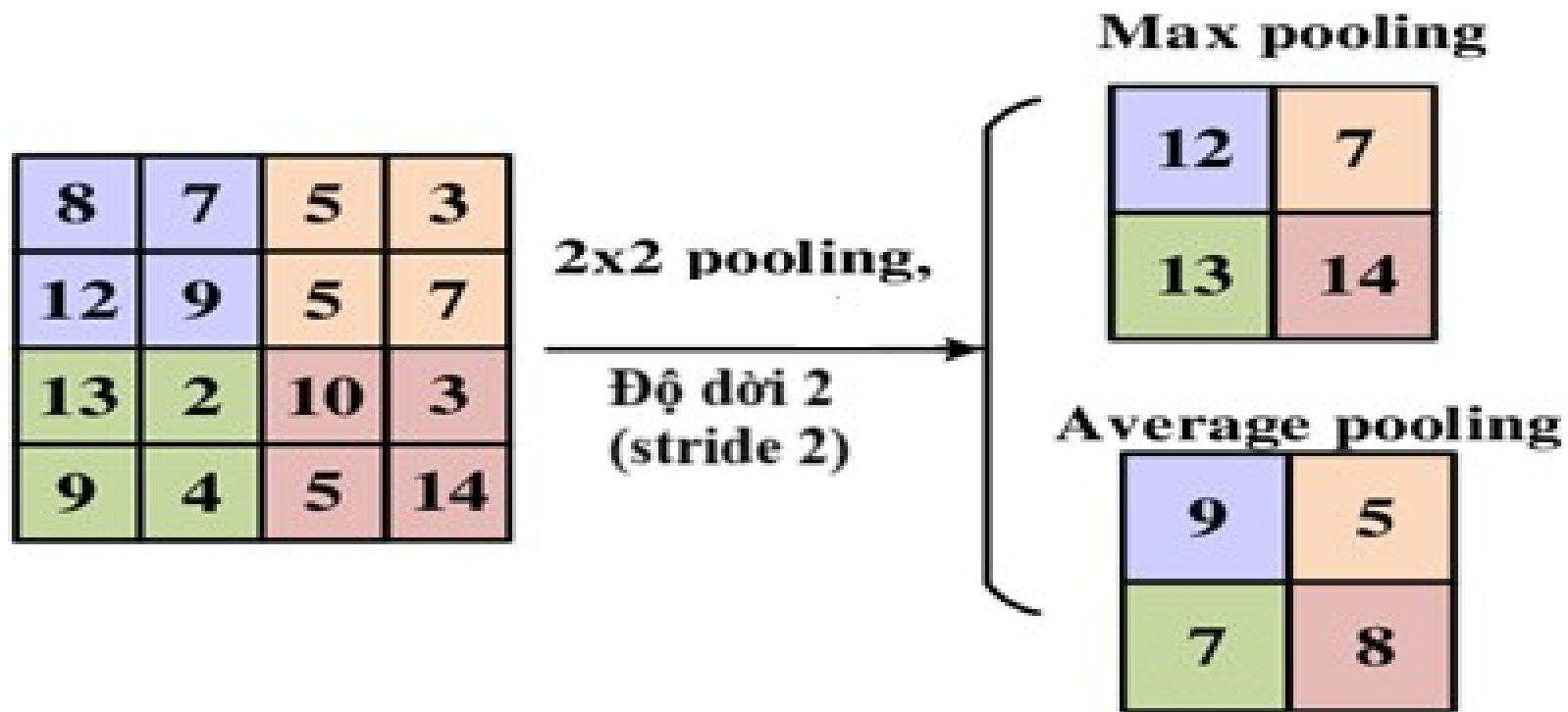
Hình 6.23: Ví dụ về cách tính nhân chập của lớp tích chập

# Ví dụ thiết kế SoC bằng phương pháp tổng hợp cấp cao



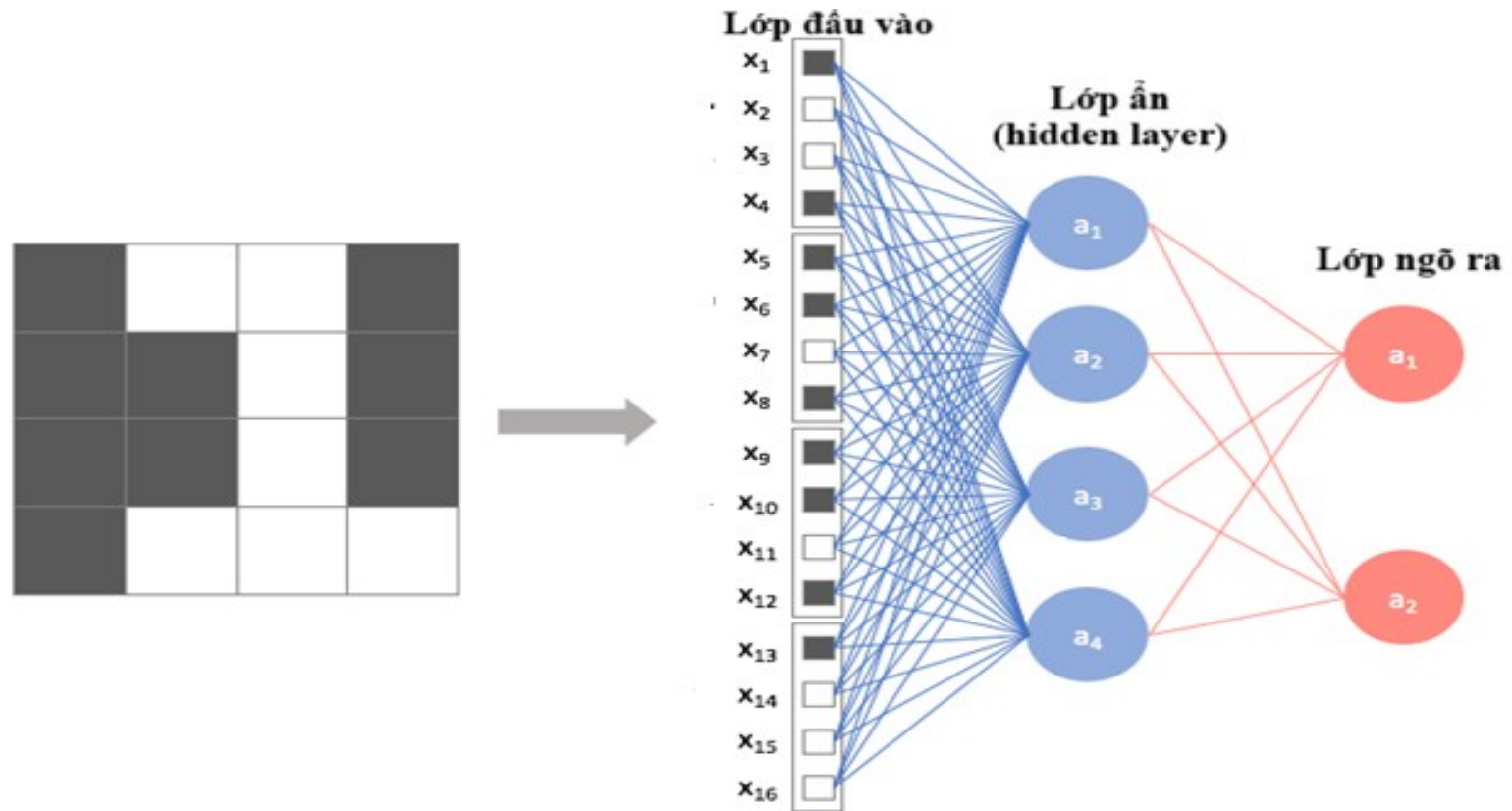
Hình 6.24: Công thức và đồ thị một số hàm kích hoạt phổ biến

# Ví dụ thiết kế SoC bằng phương pháp tổng hợp cấp cao



Hình 6.25: Ví dụ về max pooling và average pooling

# Ví dụ thiết kế SoC bằng phương pháp tổng hợp cấp cao

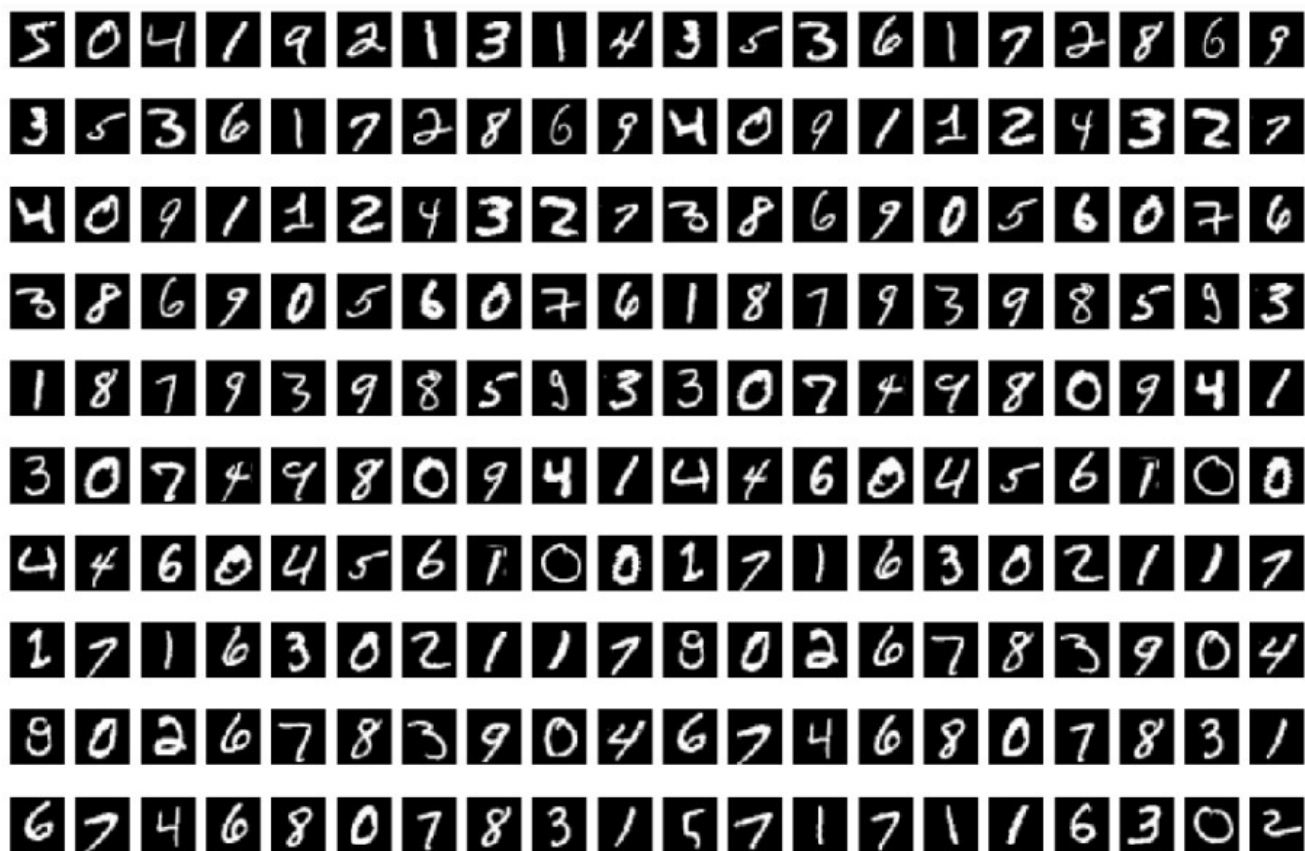


Hình 6.26: Cấu trúc lớp làm phẳng và kết nối đầy đủ



# Ví dụ thiết kế SoC bằng phương pháp tổng hợp cấp cao

*Áp dụng quy trình thiết kế SoC vào ứng dụng nhận dạng chữ số bằng AI*



Hình 6.27: Minh họa hình ảnh cụ thể trong tập dữ liệu MNIST



# Ví dụ thiết kế SoC bằng phương pháp tổng hợp cấp cao

**Bảng 6.3: Mô tả chi tiết cấu trúc mạng Lenet 5**

| Lớp                                  | Đầu vào  | Đầu ra   | Bộ lọc | Tham số | Đệm/trượt | Tổng tham số |
|--------------------------------------|----------|----------|--------|---------|-----------|--------------|
| Lớp tích chập<br>Kích hoạt<br>(ReLU) | 1x28x28  | 6x28x28  | -      | -       | 1-1       | 60           |
| Lớp gộp<br>(Max Pooling)             | 6x28x28  | 6x14x14  | 3x3    | -       | 0-2       | 0            |
| Lớp tích chập<br>Kích hoạt<br>(ReLU) | 6x14x14  | 16x10x10 | -      | -       | 0-1       | 2416         |
| Lớp gộp<br>(Max Pooling)             | 16x10x10 | 16x5x5   | 3x3    | -       | 0-2       | 0            |
| Làm phẳng                            | 16x5x5   | 400      | -      | -       | -         | 0            |

# Ví dụ thiết kế SoC bằng phương pháp tổng hợp cấp cao

| Lớp                                       | Đầu vào | Đầu ra | Bộ lọc | Tham số | Đệm/trượt | Tổng tham số |
|-------------------------------------------|---------|--------|--------|---------|-----------|--------------|
| Lớp kết nối đầy đủ<br>Kích hoạt (ReLU)    | 400     | 128    | -      | 400x128 | -         | 47120        |
| Lớp kết nối đầy đủ<br>Kích hoạt (ReLU)    | 128     | 84     | -      | 128x84  | -         | 10164        |
| Lớp kết nối đầy đủ<br>Kích hoạt (Softmax) | 84      | 10     | -      | 84x10   | -         | 850          |
| Tổng tham số                              |         |        |        |         |           | 61,610       |

# Ví dụ thiết kế SoC bằng phương pháp tổng hợp cấp cao

```
class LeNet5(nn.Module):
    def __init__(self, *args, **kwargs):
        super(LeNet5, self).__init__()
        self.cnn1 = nn.Conv2d(1, 6, (5, 5), 1) # 1 input channel, 6 output channels, 5x5 kernel
        self.pool = nn.AvgPool2d(kernel_size=(2, 2), stride=2) # Average pooling
        self.cnn2 = nn.Conv2d(6, 16, (5, 5), 1) # 6 input channels, 16 output channels, 5x5 kernel
        self.cnn3 = nn.Conv2d(16, 120, (5, 5), 1) # 16 input channels, 120 output channels, 5x5 kernel
        self.fc1 = nn.Linear(120, 84) # Fully connected layer 120 -> 84
        self.fc2 = nn.Linear(84, 10) # Fully connected layer 84 -> 10

    def forward(self, x):
        # Pad the image to maintain same dimensions
        x = F.pad(x, (2, 2, 2, 2)) # Pad 2 pixels on all sides
        x = F.relu(self.cnn1(x)) # Apply ReLU activation after first convolution
        x = self.pool(x) # Apply pooling
        x = F.relu(self.cnn2(x)) # Apply ReLU activation after second convolution
        x = self.pool(x) # Apply pooling
        x = F.relu(self.cnn3(x)) # Apply ReLU activation after third convolution
        x = x.view(-1, 120) # Flatten the layer to a vector
        x = F.relu(self.fc1(x)) # Apply ReLU activation after first fully connected layer
        x = F.softmax(self.fc2(x), dim=1) # Apply softmax at the output layer
        return x
```

Hình 6.28: Minh họa đoạn mã Python của mô hình mạng  
Lenet5 được tạo bằng nền tảng Keras

# Ví dụ thiết kế SoC bằng phương pháp tổng hợp cấp cao

```

void convolution(float in[28][28], float kernel[6][5][5], float out[6][24][24])
{
    int i,j,k,l,c;
    for(c = 0; c < 6; c++) {
        for(i = 0; i < 24; i++) {
            for(j = 0; j < 24; j++) {
                for(k = 0; k < 5; k++) {
                    for(l = 0; l < 5; l++) {
                        if (k==0 && l==0) out[c][i][j] = in[i+k][j+l] * kernel[c][k][l];
                        else out[c][i][j] += in[i+k][j+l] * kernel[c][k][l];
                    } } } } }
}

void relu_bias(float in[6][24][24], float bias[6])
{
    int i,j,c;
    for(c = 0; c < 6; c++) {
        for(i = 0; i < 24; i++) {
            for(j = 0; j < 24; j++) {
                in[c][i][j] += bias[c];
                in[c][i][j] = (in[c][i][j] < 0.0f) ? 0.0f : in[c][i][j];
            } } }
}

void maxpooling(float in[6][24][24], float out[6][12][12])
{
    int i,j,c;
    for(c = 0; c < 6; c++) {
        for(i = 0; i < 12; i++) {
            for(j = 0; j < 12; j++) {
                out[c][i][j] = in[c][i*2][j*2];
                if (in[c][i*2][j*2+1] > out[c][i][j]) out[c][i][j] = in[c][i*2][j*2+1];
                if (in[c][i*2+1][j*2] > out[c][i][j]) out[c][i][j] = in[c][i*2+1][j*2];
                if (in[c][i*2+1][j*2+1] > out[c][i][j]) out[c][i][j] = in[c][i*2+1][j*2+1];
            } } }
}
    
```

Hình 6.29: Minh họa đoạn mã C dùng để tính toán lớp tích chập, lớp kích hoạt (relu) và lớp maxpooling.

# Ví dụ thiết kế SoC bằng phương pháp tổng hợp cấp cao

| Use | Connections | Name                    | Description                    | Export                 | Clock        | Base | End | IRQ | Tags | Opcode Name |
|-----|-------------|-------------------------|--------------------------------|------------------------|--------------|------|-----|-----|------|-------------|
| ✓   |             | <b>clk_0</b>            | Clock Source                   | clk                    | exported     |      |     |     |      |             |
| ✓   |             | clk_in                  | Clock Input                    | reset                  | clk_0        |      |     |     |      |             |
| ✓   |             | clk_in_reset            | Reset Input                    | Double-click to export | [clk]        |      |     |     |      |             |
| ✓   |             | clk                     | Clock Output                   | Double-click to export | [clk]        |      |     |     |      |             |
| ✓   |             | clk_reset               | Reset Output                   | Double-click to export | [clk]        |      |     |     |      |             |
| ✓   |             | <b>nios2_gen2_0</b>     | Nios II Processor              | Double-click to export | clk_0        |      |     |     |      |             |
| ✓   |             | clk                     | Clock Input                    | Double-click to export | [clk]        |      |     |     |      |             |
| ✓   |             | reset                   | Reset Input                    | Double-click to export | [clk]        |      |     |     |      |             |
| ✓   |             | data_master             | Avalon Memory Mapped Master    | Double-click to export | [clk]        |      |     |     |      |             |
| ✓   |             | instruction_master      | Avalon Memory Mapped Master    | Double-click to export | [clk]        |      |     |     |      |             |
| ✓   |             | irq                     | Interrupt Receiver             | Double-click to export | [clk]        |      |     |     |      |             |
| ✓   |             | debug_reset_requ...     | Reset Output                   | Double-click to export | [clk]        |      |     |     |      |             |
| ✓   |             | debug_mem_slave         | Avalon Memory Mapped Slave     | Double-click to export | [clk]        |      |     |     |      |             |
| ✓   |             | custom_instructio...    | Custom Instruction Master      | Double-click to export | [clk]        |      |     |     |      |             |
| ✓   |             | <b>onchip_memory2_0</b> | On-Chip Memory (RAM or ROM)... | Double-click to export | clk_0        |      |     |     |      |             |
| ✓   |             | clk1                    | Clock Input                    | Double-click to export | [clk1]       |      |     |     |      |             |
| ✓   |             | s1                      | Avalon Memory Mapped Slave     | Double-click to export | [clk1]       |      |     |     |      |             |
| ✓   |             | reset1                  | Reset Input                    | Double-click to export | [clk1]       |      |     |     |      |             |
| ✓   |             | <b>jtag_uart_0</b>      | JTAG UART Intel FPGA IP        | Double-click to export | clk          |      |     |     |      |             |
| ✓   |             | clk                     | Clock Input                    | Double-click to export | [clk]        |      |     |     |      |             |
| ✓   |             | reset                   | Reset Input                    | Double-click to export | [clk]        |      |     |     |      |             |
| ✓   |             | avalon_jtag_slave       | Avalon Memory Mapped Slave     | Double-click to export | [clk]        |      |     |     |      |             |
| ✓   |             | irq                     | Interrupt Sender               | Double-click to export | [clk]        |      |     |     |      |             |
| ✓   |             | <b>CNL_0</b>            | ECG_CNK                        | Double-click to export | [clock_sink] |      |     |     |      |             |
| ✓   |             | avalon_slave_0          | Avalon Memory Mapped Slave     | Double-click to export | [clock_sink] |      |     |     |      |             |
| ✓   |             | reset_sink              | Reset Input                    | Double-click to export | clk_0        |      |     |     |      |             |
| ✓   |             | clock_sink              | Clock Input                    | Double-click to export | clk_0        |      |     |     |      |             |

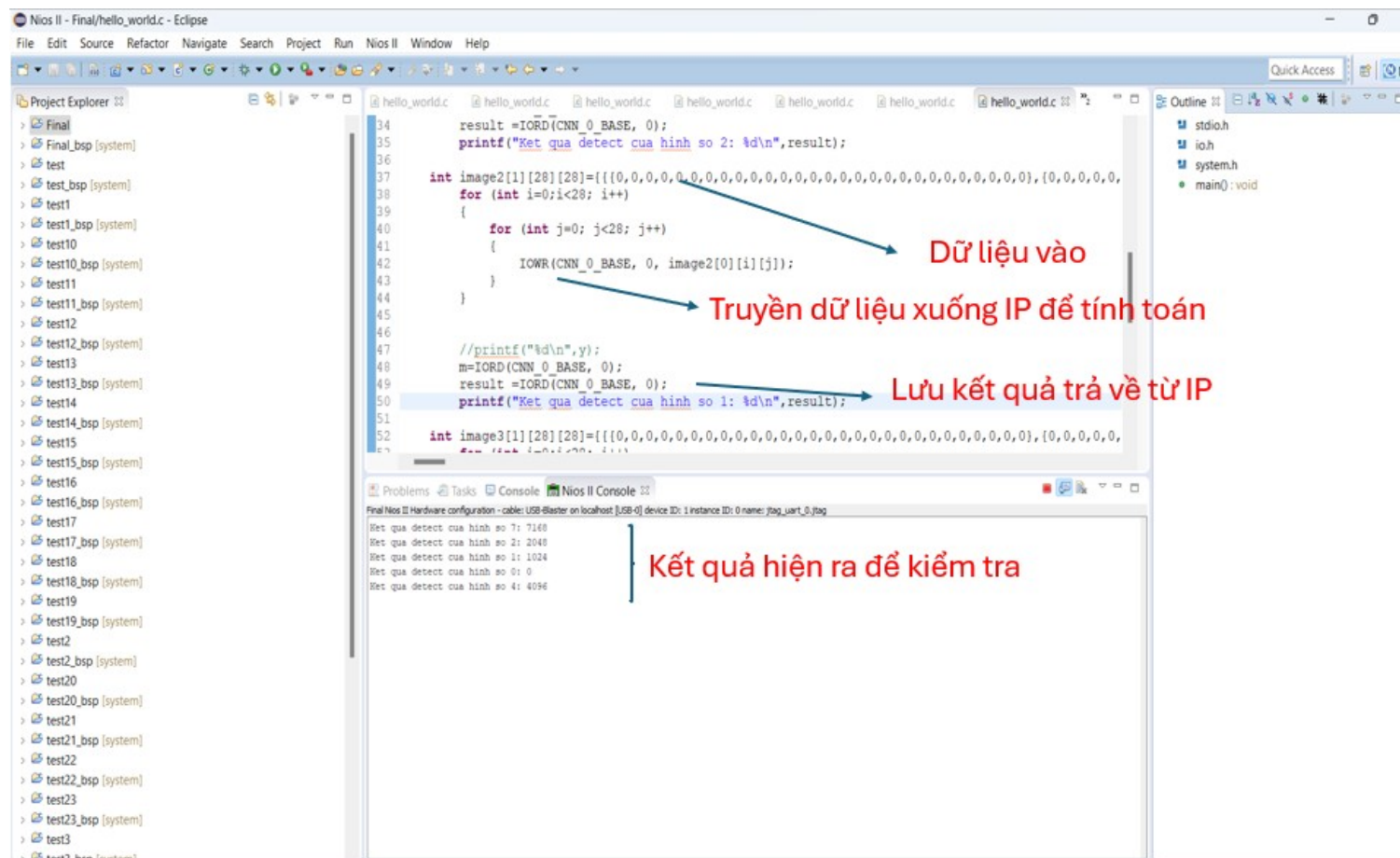
Current filter:

Messages

| Type           | Path               | Message                                                                                               |
|----------------|--------------------|-------------------------------------------------------------------------------------------------------|
| 1 Info Message | system.jtag_uart_0 | JTAG UART IP input clock need to be at least double (2x) the operating frequency of JTAG TCK on board |

Hình 6.30: Hệ thống SoC thiết kế cho ứng dụng nhận dạng chữ số trên FPGA Altera

# Ví dụ thiết kế SoC bằng phương pháp tổng hợp cấp cao



Hình 6.31: Minh họa lập trình code trên phần mềm để điều khiển và lấy kết quả từ phần cứng

# Câu hỏi và bài tập

1. Viết chương trình C thực hiện bài toán tính tổng tất cả các phần tử của một dãy số gồm N số thực. Tối ưu hóa mạch nhân sang số fixpoint 16 bit, với 5 bit độ chính xác cho phần thập phân. Dùng HLS chuyển code C sang dạng RTL. Thực hiện mô phỏng RTL code vừa tạo ra từ HLS.
2. Dựa vào thông tin tài nguyên phần cứng trích xuất được từ bài 1, cho biết thiết kế nào tốn nhiều tài nguyên phần cứng hơn
3. Thực hiện tính toán tần số FMAX, công suất tiêu thụ cho thiết kế ở bài tập 1. Với giả thuyết tần số cài đặt ban đầu là 200 Mhz.
4. Viết code phần mềm điều khiển hệ thống trên, với dữ liệu mô phỏng đầu vào là 1 chuỗi số ngẫu nhiên kiểu float.

Viết code C dùng để tính phép nhân hai ma trận  $A$  và có kích thước  $M \times N$

Viết cái testbench để test code đó. Chạy trên kiểm tra kết quả trên phần mềm code C bình thường.  
(kiểu dữ liệu là kiểu float)



```
input  ap_clk;
input  ap_rst;
input  ap_start;
output ap_done;
output ap_idle;
output ap_ready;
output [3:0] firstMatrix_address0;
output firstMatrix_ce0;
input  [31:0] firstMatrix_q0;
output [3:0] firstMatrix_address1;
output firstMatrix_ce1;
input  [31:0] firstMatrix_q1;
output [2:0] secondMatrix_address0;
output secondMatrix_ce0;
input  [31:0] secondMatrix_q0;
output [2:0] secondMatrix_address1;
output secondMatrix_ce1;
input  [31:0] secondMatrix_q1;
output [2:0] result_address0;
output result_ce0;
output result_we0;
output [31:0] result_d0;
```